

Threads in TRENTOS

- [Design considerations](#)
- [Status](#)
- [Camkes](#)
- [C code](#)

Design considerations

- the idea was to implement a statically defined thread pool by using the constructs currently available in the sel4 repos (no fork / upsteaming new features)
 - to implement thread we (ab)use the interface threads created for some connector types and write our own custom connector ([Custom connectors in TRENTOS](#))

Status

- a working POC can be found here https://bitbucket.hensoldt-cyber.systems/projects/SS/repos/demo_hello_world/browse?at=refs%2Fheads%2FSEOS-thtst
 - error handling could be improved (thread start should return error codes)
 - cancelling a thread is not possible

Camkes

A component implementing a thread pool needs to provide a few camkes resources:

- the 2 endpoints used by the connector (if_thread)
- a number of events used to start/stop the threads
- a mutex to protect shared data during thread start/stop
- semaphores for the available threads in the pool / for the return value

Resources needed in the component

```
procedure if_thread {  
  /* empty */  
};  
  
component hello_world_app {  
  control;  
  
  consumes EventX thread_start_c;  
  emits EventX thread_start_e;  
  
  consumes EventX thread_started_c;  
  emits EventX thread_started_e;  
  
  consumes EventX thread_join_c;  
  emits EventX thread_join_e;  
  
  has mutex thread_start;  
  has mutex thread_join;  
  
  has semaphore threads_available;  
  has semaphore threads_join;  
  
  provides if_thread thread1;  
  provides if_thread thread2;  
  provides if_thread thread3;  
  provides if_thread thread4;  
  
  uses if_thread thread1_1;  
  uses if_thread thread2_1;  
  uses if_thread thread3_1;  
  uses if_thread thread4_1;  
}
```

The custom connector is then used to connect the dummy endpoints from above. Each thread in the threadpool needs 2 dummy endpoints and 1 connection.

Thread connections

```
connector trentos_thread {
    from Procedures with 0 threads; // we want to have 1 thread per connection, so at one end of the connector
    we
    to Procedure; // ask that no thread is generated.
    attribute int id = 3; // just an example how to declare an attribute for the connection. Can be set
    // afterwards in the camkes configuration
}

assembly {
    composition {
        component hello_world_app main;

        connection trentos_thread \
            test1 ( \
                from main.thread1_1, \
                to main.thread1 \
            );
    }

    configuration {
        main.threads_available_value = 0;
        main.threads_join_value = 0;
    }
}
```

C code

Each interface thread started calls into the same function. Currently 0 is passed as an argument, but this value can be replaced by a configuration value of the connector (to implement a thread ID for example).

Interface run

```
int
thread_runner(int);

int /*? me.interface.name ?*/__run(void)
{
    return thread_runner(0);
}
```

The thread runner implements an infinite loop which does the following

- increments the semaphore that counts the threads available for execution
- block waiting for a thread start event (all threads block on this event and only one will be woken up when the event is posted)
- after the event happened, it signals back that it started
- it executed the function_pointer passed as argument
- if it has to return a value, it signals this to the main thread
- returns to the available state

thread_runner

```
// This skeleton is running in each thread, implementing the framework needed
// for the various thread constructs.
int
thread_runner(int badge)
{
    while (1)
    {
        // Local variables to save the parameters passed by the create call.
        thread_func_ptr_t local_thread_to_run = NULL;
        bool               local_detached     = false;
        // Return value.
        int                return_value      = 0;

        // Post to semaphore, increasing the count of free threads in the pool
        // by one.
        if(threads_available_post())
        {
            printf("Semaphore error\n");
            return -1;
        }

        // Wait for a start request from thread_start.
        thread_start_c_wait();

        // Copy parameters passed from create to local variables, allowing
        // the global variables to be used in the next create call.
        local_thread_to_run = thread_to_run;
        local_detached      = thread_to_run_detached;
        thread_to_run       = NULL;

        // signal to thread_create that the requested thread started.
        thread_started_e_emit();

        // Check if we got an actual function pointer to execute.
        if (local_thread_to_run)
        {
            // Execute the passed function.
            return_value = local_thread_to_run();
        }

        // Check if its a JOINABLE thread which has to return a value.
        if (!local_detached)
        {
            // Lock the return value variable, to prevent other threads from
            // overwriting it.
            if (thread_join_lock())
            {
                printf("Mutex error\n");
                return -1;
            }
            // Write the return value.
            thread_return_value = return_value;
            // Signal the thread_join there is a return value to be read.
            if(threads_join_post())
            {
                printf("Semaphore error\n");
                return -1;
            }
        }
    }

    return 0;
}
```

Thread control functions

```
typedef int (*thread_func_ptr_t)(void);

// Parameters passed to the thread that will be started
volatile thread_func_ptr_t thread_to_run = NULL;
volatile bool thread_to_run_detached = false;

// Return value from the thread being joined
volatile int thread_return_value = 0;

// Used to execute the function passed as an argument on one thread from the
// thread pool in either joinable or detached fashion.
void
thread_start(thread_func_ptr_t func, bool detached)
{
    // Get a mutex to guarantee only one instance of this function is active at
    // a given time.
    if (thread_start_lock())
    {
        printf("Mutex error\n");
        return;
    }

    // Check on the semaphore if there is a free thread in the thread pool.
    // In place of busy waiting we can return an out of resources / try again
    // error and let the user deal with it.
    // If there is a free thread we reserve it here.
    while (threads_available_trywait())
    {
        printf("Waiting for a free thread in the pool\n");
        seL4_Yield();
    }

    // Prepare the arguments to be passed to the thread we start.
    thread_to_run = func;
    thread_to_run_detached = detached;

    // Wake up a random thread from the pool.
    thread_start_e_emit();
    // Wait for confirmation it started executing.
    thread_started_c_wait();

    // Unlock the mutex, allowing the next start call to be executed.
    if (thread_start_unlock())
    {
        printf("Mutex error\n");
        return;
    }
}

// Wait for a joinable thread to finish and get its return code.
int
thread_join()
{
    // Wait for a thread to finish.
    // Could be made non-blocking by using trywait
    if(threads_join_wait())
    {
        printf("Semaphore error\n");
        return -1;
    }
    // Get its return value.
    int return_value = thread_return_value;
    // Release the mutex signaling we read the return value.
    if (thread_join_unlock())
    {
        printf("Mutex error\n");
        return -1;
    }
}
```

```
    }  
    return return_value;  
}
```